ANALISIS TECNICO COMPLETO

Centro COP Escalable y migracion a Spring Boot

Inventario de servicios, arquitectura, seguridad, contratos y plan de migracion

Fecha de revision: 31 de agosto de 2026

Fuente: analisis actualizado del repositorio y alcance tecnico vigente.

Fecha de revisión: 31 de agosto de 2026

Alcance: inventario del repositorio, servicios desplegables, contratos HTTP y estado real de migración.

Decisión aplicada: se retiraron web-public/ y web-dashboard/ (Next.js), sus builds de CI y sus dos servicios Render. No contenían rutas API, acceso directo a MongoDB, Server Actions, middleware de negocio ni ORM; eran consumidores HTTP de la API. Por tanto no existe código de servidor de Next.js que trasladar literalmente a Java.

1. Resumen ejecutivo

Centro COP es una plataforma clínica multitenant para odontología y psicología. Gestiona identidad, pacientes, agenda, servicios, historia clínica, psicología, odontograma, reservas públicas, pagos, analítica, predicción J48 e integraciones de IA/imagen 3D.

La base de datos operativa es MongoDB; Redis respalda colas, caché y tareas asíncronas. Actualmente NestJS conserva la mayor parte del dominio. backend-spring ya implementa un primer corte funcional en Java 21/Spring Boot 3.4: salud, autenticación, pacientes, agenda, catálogo y sedes. services/j48-service es un segundo servicio Spring Boot independiente que entrena y consulta modelos Weka J48.

La retirada de Next.js reduce dos runtimes Node de presentación, pero no equivale a retirar NestJS: el código de negocio pendiente está en nest-migration/ y en servicios Python especializados. La sustitución debe hacerse por módulos, con compatibilidad de contratos y pruebas de regresión, no con stubs.

2. Arquitectura tras la retirada de Next.js

Navegador

├── Frontend/ Angular: panel clínico

└── PublicWeb/ Angular: sitio público y reservas

│ HTTPS / JSON / WebSocket

▼

gateway nginx :8080

│

├── NestJS :8080 (módulos aún no migrados)

├── Spring COP API :8081 (módulos migrados; aún no conectado al gateway)

├── J48 Spring/Weka :8080 interno, perfil j48-java

└── FastAPI/ML y 3D (transitorios)

│

▼

MongoDB Atlas / Redis

El gateway sigue apuntando a NestJS. No se modificó para dirigir tráfico a backend-spring, porque Spring todavía no cubre todos los contratos. El corte se realizará ruta a ruta al terminar cada módulo y sus pruebas de caracterización.

3. Componentes e infraestructura

Componente	Tecnología	Responsabilidad	Estado
Frontend/	Angular	Panel administrativo y clínico	Se conserva como interfaz interna
PublicWeb/	Angular	Portal, catálogo, autenticación y reservas	Se conserva como interfaz pública
backend-spring/	Java 21, Spring Boot, MongoDB	Reemplazo incremental del API de negocio	Activo; cobertura parcial

nest-migration/	NestJS, Mongoose	API transaccional heredada y módulos pendientes	Activo durante coexistencia
services/gateway/	Nginx	Entrada HTTP y proxy de compatibilidad	Activo; apunta a Nest
services/j48-service/	Spring Boot, Weka	Entrenamiento, inferencia y explicación J48	Activo como servicio separado
services/j48-python/	FastAPI, scikit-learn	Motor J48 alternativo usado por Nest	Transitorio; validar paridad con Weka
recommendation-engine	FastAPI/ML	Recomendación y riesgo de recaída	Transitorio
ai-diagnosis-service	FastAPI/ML	Diagnóstico asistido	Transitorio
emotion-analysis-service	FastAPI/ML	Análisis de emoción	Transitorio
image-to-3d-stub / image-to-3d-depth	FastAPI, Celery	Imagen a malla 3D; el stub no es producción	Transitorio
dicom-to-glb	Python	DICOM/CBCT a GLB	Transitorio
ortho_system	Python	Procesamiento ortodóntico/IA	Transitorio
MongoDB	Documento	Persistencia operacional y colecciones clínicas	Fuente de verdad actual
Redis	Key-value	Caché, coordinación y colas	Dependencia operacional

4. Seguridad, datos y reglas no negociables

1. Autenticación: Bearer JWT HS256 con JWT_SECRET; acceso de duración corta y refresh token rotativo almacenado en refresh_tokens.

2. Contraseñas: bcrypt; no se debe cambiar el formato de hash al migrar usuarios.

3. Autorización: roles SUPER_ADMIN, ADMIN, ORG_ADMIN, SITE_ADMIN, MEDICO y PROFESSIONAL, con filtrado por organización y sede.

4. Multitenancy: toda lectura/escritura clínica debe restringirse por organization_id y, cuando aplique, site_id. No basta con filtrar en el frontend.

5. MongoDB: hay identificadores UUID serializados como string y valores BSON. Repositorios Spring deben aceptar ambas representaciones durante la transición.

6. Datos sensibles: historia, diagnósticos, predicciones, pagos y auditaría requieren mínimo privilegio, trazabilidad y nunca deben aparecer en logs de aplicación.

7. Integridad transaccional: creación de cita, bloqueo de solapamientos, confirmación de pago y procesamiento de webhook deben ser idempotentes.

Colecciones detectadas: organizations, sites, users, refresh_tokens, professionals, patients, appointments, clinical_records, odontograms, psychology_sessions, psychological_evaluations, psychological_snapshots, j48_predictions, medical_ai_*, ortho_3d_jobs, catalog_services, service_offerings, service_categories, public_reviews y datos de reserva/pago.

5. Cobertura actualmente implementada en Spring Boot

Dominio	Endpoints Spring	Reglas preservadas	Estado
Salud	GET /health, GET /health/live	Liveness y disponibilidad del proceso	Migrado
IAM	POST /api/auth/login, POST /api/auth/refresh	bcrypt, JWT, rotación de refresh	Migrado parcialmente; faltan registro, Google, logout, perfil y administración
Pacientes	GET/POST /api/patients	Roles, organización/sede, búsqueda y paginación	Migrado

Agenda	GET/POST /api/appointments, GET /professionals, PATCH /{id}/status, PATCH /{id}/claim	Roles, profesional, estados y solapamiento	Migrado
Catálogo	CRUD /api/services y categoría	Catálogo, ofertas y categorías por tenant	Migrado
Sedes	GET /public/departments, /public/sites, /api/sites, /api/sites/departments	Visibilidad pública y administrativa	Migrado parcialmente; falta administración/sincronización

backend-spring usa Spring Web, Validation, Security, Spring Data MongoDB, Actuator, OpenAPI y JJWT. Su documentación interactiva es /api/docs.

6. Inventario funcional completo y plan por módulo

Módulo fuente	Contratos principales	Dependencias	Destino Spring Boot / prioridad
IAM	/api/auth/register, google, login, logout, refresh, /api/users/me, /api/admin/users	Mongo, bcrypt, JWT	Extender iam; P0. Añadir registro, logout/revocación, perfil, roles y bootstrap seguro.
Pacientes	/api/patients	Mongo, tenant/RBAC	patients; migrado, completar pruebas de contrato.
Agenda	/api/appointments, profesionales, estado, claim	Mongo, profesional, notificaciones	appointments; migrado, añadir pruebas de concurrencia para solapamiento.
Catálogo	/api/services CRUD y categoría	Mongo, tenant/RBAC	catalog; migrado, validar forma exacta de respuesta.
Sedes/tenancy	/public/departments, /public/sites, /api/sites, sync-catalog	Mongo	tenancy; rutas de consulta migradas, CRUD/sync P1.
Historia clínica	/api/clinical/records/{patientId} y entradas	Mongo, paciente, auditoría	Crear clinical; P0. Versionar entradas y validar tenant.
Odontograma	GET/PATCH /api/odontogram/{patientId}	Mongo, historia	Crear odontogram; P1. Mantener semántica de actualización parcial.
Odontología	planes y sugerencias de plan	Mongo, IA	Crear odontology; P1. Separar sugerencia de decisión clínica final.
Psicología	escalas, DSM, sesiones, evaluaciones y evolución	Mongo, profesional	Crear psychology; P0 por información clínica sensible.
Pruebas psicológicas	templates y submissions	Mongo, IA	Integrar en psychology; P1.
J48 scoring	score individual/masivo, analíticas, historial	Mongo, J48 HTTP	Crear j48; P0. Consumir primero j48-service con timeout, validación y trazabilidad.
Laboratorio Weka	datasets, modelos, árbol, comparar, predecir	Mongo, archivos, Weka	Crear weka-lab; P1. Reutilizar el microservicio Java y aislar carga de archivos.
IA médica	dashboard, alertas, evaluación, timeline, asistente, insights	Mongo, Redis, IA externa, WebSocket	Crear medical-ai; P1. Usar cliente HTTP resiliente y cola; no bloquear request clínico.

Recaída	riesgo, tendencia, acknowledge/assess	Mongo, recommendation engine	Integrar en medical-ai; P1, con fallback explícito y métricas de modelo.
IA assist/coplan	contexto, sugerencias, aprobar/rechazar	Mongo, LLM/IA	Crear ai-assist; P2. Toda salida requiere revisión humana.
Portal paciente	token, autenticación, dashboard, timeline, tratamientos, citas	Mongo, JWT de portal	Crear portal; P2. Tokens de alcance mínimo.
Presupuestos y decisiones	generar, aprobar, simulación, recomendaciones	Mongo	Crear budget y decisions; P1. Idempotencia y auditoría.
Seguimiento y terapia	encuestas, schedules, sesiones, módulos y progreso	Mongo, notificaciones	Crear followup/therapy; P2.
Experiencia/personalización	perfiles, recomendaciones, encuestas, churn	Mongo, ML	Crear experience; P2.
Sitio público y reservas	catálogo, disponibilidad, cotización, booking, confirmación	Mongo, SMTP/Twilio	Crear publicsite; P0 antes de apagar Nest.
Pagos Wompi	intents, complete, webhook, presets/context	Wompi, Mongo	Crear payments; P0. Firma, idempotencia y reconciliación obligatorias.
Pagos Stripe/PayPal	métodos, intent/order	APIs externas	Integrar en payments; P1.
Reseñas públicas	lectura, envío, moderación	Mongo, RBAC	Integrar en publicsite; P2.
Notificaciones	reserva, email/WhatsApp	SMTP, Twilio, Redis	Crear notifications; P1. Usar outbox/cola, reintentos y deduplicación.
Analítica	KPIs, tendencias, distribución, heatmap	Mongo aggregations	Crear analytics; P2. Comparar resultados de agregaciones.
Simulación	historial y detalle de simulaciones	Mongo	Crear simulation; P2.
Ortho 3D y rayos X	reconstrucción, DICOM, jobs, GLB	proveedores 3D, archivos, Redis	Crear ortho3d; P1. Job asíncrono, límites MIME/tamaño, URLs firmadas.
Proxy AI/compatibilidad	árbol J48 y endpoints de transición	J48/Python	No copiar stubs: sustituir cada contrato por un módulo real; P1/P2.

7. Servicios especializados: tratamiento en Java

Servicio actual	**Tratamiento objetivo**
j48-service	Mantener y consolidar como servicio Java. Entrena desde ARFF, persiste el modelo, expone inferencia y tiene pruebas. Publicar un contrato versionado antes de que Spring COP lo consuma.
j48-python	No eliminar hasta comparar clasificación, probabilidad, árbol y explicación contra el J48 Java usando el mismo ARFF y casos borde. Si hay paridad aceptada, retirar del gateway y Compose.
Diagnóstico, emoción y recomendación	Mantener como inferencia especializada detrás de adaptadores Spring WebClient/HTTP con timeouts, circuit breaker, métricas y correlación. Migrar a ONNX

	Runtime Java o DJL sólo cuando el artefacto/modelo y las pruebas de paridad lo permitan.
Imagen 3D/DICOM/ortho	Mantener workers Python: su stack científico no es una simple conversión a Java. Spring debe orquestar trabajos, autorización, persistencia y descarga; los workers generan el resultado. El image-to-3d-stub no es válido en producción.
Redis/colas	Sustituir el uso de BullMQ con un mecanismo explícito (Redis Streams, RabbitMQ o cola gestionada), con idempotency key, DLQ, reintentos y observabilidad.

8. Estrategia de corte seguro

1. Congelar y exportar un inventario de requests/responses de Nest para cada módulo. Incluir códigos de estado, paginación, validaciones, roles y errores.

2. Implementar DTOs Java validados y servicios Spring contra una copia anonimizada de MongoDB.

3. Crear pruebas de contrato Nest↔Spring y pruebas de integración con Mongo/Redis. Para pagos y clínica, usar también pruebas de idempotencia y autorización cruzada de tenants.

4. Montar una ruta canary en Nginx para el módulo terminado. No enviar tráfico de módulos no migrados a Spring.

5. Comparar métricas, logs de error y datos escritos; disponer de rollback inmediato al upstream Nest.

6. Una vez validado el conjunto de módulos, cambiar el gateway a Spring y retirar NestJS, los servicios Python que tengan equivalencia demostrada y las variables/configuración obsoletas.

9. Validación de la retirada de Next.js

Se eliminaron los directorios web-public/ y web-dashboard/, incluidos node_modules y .next. También se retiraron:

- Los dos jobs Next.js de .github/workflows/ci.yml; se añadió spring-api, que ejecuta mvn verify con Java 21.

- Los dos servicios Node de render.yaml y sus URLs de CORS asociadas.

- Las referencias de inicio rápido y componentes en el README.md.

Los proyectos Angular permanecen porque sustituyen la capa de presentación. Sus llamadas deben apuntar gradualmente a las rutas Spring que hayan alcanzado paridad. La migración es del backend de dominio NestJS a Spring Boot, no una transpilación de componentes de interfaz a Java.

10. Riesgos y criterios de aceptación

Riesgo	**Control requerido**
Fuga entre organizaciones/sedes	Pruebas de aislamiento por endpoint y filtros obligatorios en repositorio/servicio.
Pérdida o duplicado de pago	Webhooks firmados, idempotency key, registro de eventos y reconciliación.
Cambio silencioso del modelo ML	Dataset de regresión, umbrales acordados, versión de modelo y auditoría de inferencia.
Diferencias Mongo/Mongoose	Pruebas sobre documentos reales con UUID string/BSON y agregaciones equivalentes.
Caída de proveedor IA/3D	Timeout, circuit breaker, cola y estado de job consultable; no devolver éxito ficticio.
Secretos expuestos	Rotar secretos de desarrollo existentes, eliminar archivos temporales y usar secretos del proveedor de despliegue.

La migración se considera terminada únicamente cuando todas las rutas activas tienen implementación Spring, cobertura de contratos aprobada, monitoreo operativo, rollback probado y el gateway deja de depender de NestJS.